

Раздел 3. Базы данных

Содержание:

1. Реляционная модель данных: Основные понятия: отношение, атрибут, кортеж, домен, схема, степень. Ключи: потенциальный, первичный, внешний.
2. Основы SQL. SELECT: синтаксис, порядок выполнения. Фильтрация, сортировка, агрегация. Соединений (JOIN), типы соединений. Теоретико-множественные операции.
3. Оконные функции в SQL: синтаксис OVER(), PARTITION BY, ORDER BY. агрегатные функции в оконном контексте. Ранжирующие функции. Смещения. Фреймы.
4. CTE (Common Table Expressions): Синтаксис WITH, рекурсивные CTE.
5. Проектирование баз данных. Обеспечение целостности, устранение аномалий, минимизация избыточности. Этапы проектирования: Концептуальное (ER-диаграммы, связи), логическое (нормализация), физическое.
6. Нормализация в базах данных. Нормальные формы: определения 1НФ, 2НФ, 3НФ, НФ Бойса-Кодда.
7. Транзакции в SQL. Команды для работы с транзакциями. Свойства ACID: Атомарность, Согласованность, Изолированность, Долговечность.
8. Уровни изоляции транзакций. Проблемы изолированности: "Грязное чтение", "Неповторяющееся чтение", "Фантомы".
9. План запроса: Операторы EXPLAIN и ANALYZE. Чтение и анализ плана (Seq Scan, Index Scan, Join методы).
10. Индексы: Назначение, условия использования. Типы: B-Tree, Hash, GiST, GIN, BRIN (номинально)
11. Методы доступа к данным: Seq Scan, Index Scan, Index Only Scan, Bitmap Scan.
12. Буферный кэш и WAL (Write-Ahead Log): Принципы работы, роль в обеспечении ACID.
13. Реализация изоляций транзакций в PostgreSQL. Версии строк. Снимки данных. Отмена транзакций. Горизонт транзакции и горизонт базы данных. Ручная и автоматическая очистка (VACUUM).

Введение (Главные определения раздела):

def База данных (БД) – это:

- совокупность данных, хранимых в соответствии со схемой данных, манипулирование которыми выполняют в соответствии правилами средств моделирования данных.
- Некоторый набор перманентных (постоянно хранимых) данных, используемых прикладными программными системами какого-либо предприятия.

def Система Управления БД (СУБД) – это совокупность программных и лингвистических средств, обеспечивающих управление созданием и использованием БД.

1. Реляционная модель данных: Основные понятия: отношение, атрибут, кортеж, домен, схема, степень. Ключи: потенциальный, первичный, внешний.

def Модель данных – это

- Абстрактное, самодостаточное, логическое определение объектов, операторов и прочих элементов, в совокупности составляющих абстрактную машину доступа к данным, с которой взаимодействует пользователь.
- Инструмент, формальная теория представления и обработки данных в СУБД, включающая по меньшей мере три аспекта:
 - Аспект структуры: методы описания типов и логических структур данных в БД;
 - Аспект манипуляции: методы манипулирования данными;
 - Аспект целостности: методы описания и поддержки целостности БД.

Реляционная модель данных

ЗАГЛОВОК										
ОТНОШЕНИЕ										
СТЕПЕНЬ: 10										
АТТРИБУТ										
КОРТЕЖ										
ТЕЛО										
ДОМЕН:										
"NEAR BAY", "ON OCEAN", "INLAND", "NEAR OCEAN", "ISLAND"										
longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity	
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY
...	...	...	...	...	...	...	...	...	...	...
20635	-121.09	39.48	25.0	1665.0	374.0	845.0	330.0	1.5603	78100.0	INLAND
20636	-121.21	39.49	18.0	697.0	150.0	356.0	114.0	2.5568	77100.0	INLAND
20637	-121.22	39.43	17.0	2254.0	485.0	1007.0	433.0	1.7000	92300.0	INLAND
20638	-121.32	39.43	18.0	1860.0	409.0	741.0	349.0	1.8672	84700.0	INLAND
20639	-121.24	39.37	16.0	2785.0	616.0	1387.0	530.0	2.3886	89400.0	INLAND

20640 rows x 10 columns

def Отношение – математическая структура, которая формально определяет свойства различных объектов и их взаимосвязи. (Проще: табличка с данными, в которой не определен порядок строк и столбцов, а также нет двух одинаковых строк)

def Реляционная алгебра – коллекция операций, которые принимают отношения в качестве операндов и возвращают отношение в качестве результата. (Проще: набор

функций, принимающих на вход и возвращающих табличку (отношение))

def Домен – это множество допустимых значений.

def Атрибут – наименование домена. (*Проще: название колонки*)

def Кортеж – упорядоченный набор фиксированной длины. (*Проще: строка таблицы*)

def Арность отношения (степень) – количество атрибутов отношения.

def Заголовок отношения – список атрибутов.

def Тело отношения – множество кортежей в составе отношения.

Ключи

def Потенциальный ключ (Candidate key, СК) – это подмножество атрибутов отношения, удовлетворяющее требованиям уникальности и минимальности:

- Уникальность: нет и не может быть двух кортежей данного отношения, в которых значения этого подмножества атрибутов совпадают.
- Минимальность: в составе потенциального ключа отсутствует меньшее подмножество атрибутов, удовлетворяющее условию уникальности. (Не можем из имеющегося потенциального ключа вычеркнуть атрибут и вновь получить потенциальный ключ)

def Первичный ключ (Primary key, РК) – это один из потенциальных ключей отношения, выбранный в качестве основного.

- Если в отношении имеется лишь один СК, он и будет первичным ключом. В противном случае один ключ выбирается в качестве первичного, а остальные называются альтернативными.
- Отсутствие значения для РК недопустимо.
- В теории, все потенциальные ключи одинаково пригодны для использования в качестве РК, но на практике используется СК, занимающий меньше места при хранении и который не утратит свою уникальность со временем.

def Суррогатный ключ – это дополнительное служебное поле, которое добавляется к уже имеющимся информационным полям таблицы только чтобы служить первичным ключом.

- Значение генерируется искусственно и смысловой нагрузки значение не несёт.
- Обычно является числовым полем и генерируется за счет автоинкремента. (Для PostgreSQL это реализовано в типе данных `SERIAL`)

Достоинства суррогатных ключей	Недостатки суррогатных ключей
<i>Неизменность</i> : заполнили одним значением раз и навсегда (кроме экстраординарных ситуаций).	<i>Уязвимость генераторов</i> : по номерам ключей возможно узнать число новых записей за определенный период времени.
<i>Гарантированная уникальность</i> : т.к. значение генерируется автоинкрементом, повторение значений исключено.	<i>Неинформативность</i> : усложняется ручная проверка БД.
<i>Гибкость</i> : т.к. такой ключ не несет никакой информативной нагрузки, его можно свободно заменить.	<i>Склоняет к пропуску нормализации</i> : декомпозиции предпочитают создание суррогатного ключа.
<i>Эффективность</i> : удобнее при создании ссылок на другие таблицы.	<i>Привязка разработчика к поведению генератора ключей в конкретной СУБД</i> .

def Пусть R_1 и R_2 – две переменные отношения, необязательно различные. **Внешним ключом (Foreign Key, FK)** в R_2 является подмножество атрибутов переменной R_1 такое, что выполняются следующие требования:

- В переменной отношения R_1 имеется потенциальный ключ РК такой, что РК и FK совпадают с точностью до переименования атрибутов.
- В любой момент времени множество всех значений FK в R_2 является подмножеством значений РК в R_1 .

то есть:

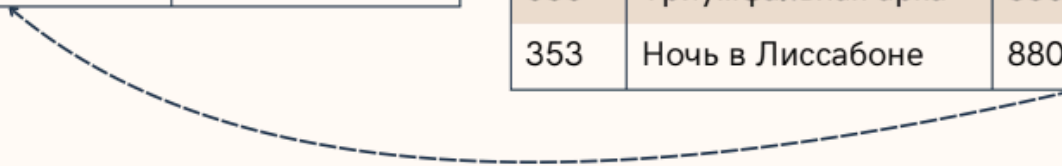
- R_1 содержит потенциальный ключ и является главным, целевым или родительским отношением.
- R_2 содержит внешний ключ (Foreign Key, FK) и называется подчиненным или дочерним отношением.

родительское отношение

author_id (PK)	author_name
1111	Джордж Оруэлл
1024	Олдос Хаксли
8800	Эрих Мария Ремарк

дочернее отношение

id (PK)	book_name	author_id (FK)
924	1984	1111
822	Гений и богиня	1024
215	О дивный новый мир	1024
535	Триумфальная арка	8800
353	Ночь в Лиссабоне	8800



def Ссылочная целостность – это необходимое качество реляционной БД, заключающееся в отсутствии в любом её отношении внешних ключей, ссылающихся на несуществующие кортежи.

2. Основы SQL. SELECT: синтаксис, порядок выполнения. Фильтрация, сортировка, агрегация. Соединений (JOIN), типы соединений. Теоретико-множественные операции.

Основы SQL

def SQL (Structured Query Language) – декларативный язык программирования, применяемый для создания, модификации и управления данными в реляционной БД, управляемой соответствующей СУБД.

Декларативный = описывается ожидаемый результат, а не способ его получения.

Виды операторов в SQL:

- **Data Definition Language (DDL):** CREATE , ALTER , TRUNCATE , DROP
- **Data Manipulation Language (DML):** SELECT , INSERT , UPDATE , DELETE
- **Data Control Language (DCL):** GRANT , REVOKE
- **Data Transaction Language (DTL):** COMMIT , ROLLBACK

{skippable} Типы данных в SQL

Типы данных:

- Целочисленные:

- `SMALLINT` – знаковое 2-байтное целое число $[-32768, 32767]$
- `INTEGER` – знаковое 4-байтное целое число $[-2147483648, 2147483647]$
- `BIGINT` – знаковое 8-байтное целое число $[-2^{63}, 2^{63} - 1]$
- Числа с произвольной точностью `NUMERIC(p [, s])` (`= DECIMAL(p [, s])`):
 - Числа с заданной точностью `p` – максимальным число хранимых десятичных разрядов (слева и справа включительно).
 - Опциональный параметр `s` (гамма, scale) задаёт максимальное число разрядов *справа* от десятичной запятой. Может быть указан, только если есть `p` и должен быть $0 \leq s \leq p$.
 - С ростом `p` увеличиваются затраты на хранение, например при $p \in [29, 38]$ требует 17 байт.
- Числа с плавающей точкой:
 - `REAL ( FLOAT4 )` – 4-байтное число с плавающей точкой.
 - `DOUBLE PRECISION , ( FLOAT8 )` – 8-байтное число с плавающей точкой.
- `BOOLEAN` – `TRUE` или `FALSE`
- Символьные типы данных:
 - **`VARCHAR(n)`** – строка переменной длины с ограничением.
 - **`CHAR(n)`** – строка фиксированной длины, (дополняется ведущими пробелами).
 - **`TEXT`** – строка переменной неограниченной длины.
- Типы данных даты и времени:
 - `DATE` - только дата
 - `TIME` - только время
 - `TIMESTAMP` - дата и время
 - `TIME WITH TIMEZONE / TIMETZ` - время с учётом часового пояса
 - `TIMESTAMP WITH TIMEZONE / TIMESTAMPTZ` - дата и время с учётом часового пояса
 - `INTERVAL` - временной интервал

{skippable} DDL операторы

Оператор	Описание
<code>CREATE</code>	Оператор создания объектов БД
<code>ALTER</code>	Оператор модификации объектов БД
<code>TRUNCATE</code>	Оператор удаления всего содержимого объекта БД
<code>DROP</code>	Оператор удаления объектов БД

Примеры:

```

CREATE SCHEMA IF NOT EXISTS user_data;

/*
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] table_name (
    col_name_1    datatype    constraints,
    col_name_2    datatype    constraints,
    ...
    col_name_N    datatype    constraints
);
*/

CREATE TABLE IF NOT EXISTS user_data.users (
    user_id        UUID                PRIMARY KEY,
    email          TEXT                UNIQUE NOT NULL,
    enc_pass       TEXT                NOT NULL,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

ALTER TABLE user_data.users
ADD CONSTRAINT check_email_format CHECK (
    email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
);

TRUNCATE TABLE users_data.users;

DROP TABLE IF EXISTS users_data.users

```

***{skippable}* DML операторы (кроме SELECT)**

Оператор	Описание
INSERT	Оператор добавления новых записей
UPDATE	Оператор изменения существующих записей
DELETE	Оператор удаления записей по условию

Примеры:

```

/*
INSERT INTO table_name (column1, column2, ..., columnM)
VALUES
    (Avalue1, Avalue2, ..., AvalueM),
    (Bvalue1, Bvalue2, ..., BvalueM),
    ...

```

```

(Nvalue1, Nvalue2, ..., NvalueM);
*/

INSERT INTO users_data.users (user_id, email, enc_pass)
VALUES
    ("1ec3e839-5c60-4929-885d-481ff1d2280d", "somemail@mail.ru", "..."),
    ("2e0b614d-1015-4e2a-b5bd-019e33db8a54", "somemail@gmail.com", "..."),
    ("f1f01166-de24-45f7-99e2-f1504cb424db", "somemail@phystech.edu", "...");

/*
UPDATE table_name
    SET column1 = value1, ..., columnN = valueN
    [WHERE condition];
*/

UPDATE users_data.users
    SET enc_pass = "..."
    WHERE email = "somemail@gmail.com"

/*
DELETE
    FROM table_name
    [WHERE condition];
*/

DELETE
    FROM users_data.users
    WHERE email LIKE "%@phystech.edu"

```

Оператор SELECT

Оператор `SELECT` осуществляет выборку данных по условию.

Синтаксис:

```

SELECT [DISTINCT] select_item_comma_list
FROM table_reference_comma_list
    [WHERE conditional_expression]
    [GROUP BY column_name_comma_list]
    [HAVING conditional_expression]
    [ORDER BY order_item_comma_list];

```

Порядок выполнения:

1. `FROM` (и `JOIN`) – определяются таблицы и связываются источники.

2. `WHERE` – фильтруются исходные строки по заданным условиям.
3. `GROUP BY` – оставшиеся строки объединяются в группы.
4. `HAVING` – фильтруются уже сгруппированные данные.
5. `SELECT` – вычисляются выражения, создаются псевдонимы (алиасы) и формируется итоговый набор колонок. (Если с `DISTINCT`, то после `SELECT` удаляются дубликаты из результирующего набора)
6. `ORDER BY` – сортирует строки.
(И в конце `LIMIT` / `OFFSET` / `TOP` для ограничения количества возвращаемых строк)

SELECT: Фильтрация (WHERE)

Оператор `WHERE` принимает на вход одну таблицу и порождает таблицу с теми же атрибутами, что и входная и в которой все строки соответствуют условию фильтрации. Если во входной таблице ни одна строка не соответствует условию, то на выходе будет пустая таблица.

Условия фильтрации могут состоять из:

- Операторов сравнения:

Важное замечание: В SQL работает тернарная логика: на выходе операторов сравнения, помимо `TRUE` и `FALSE`, мы можем получить `UNKNOWN` в ситуациях, когда один из операндов `NULL`.

- `=` – Равенство
- `<=>` – Эквивалентность (Аналогичен равенству, но при сравнении `NULL` с `NULL` вернёт `TRUE`, а при сравнении `NULL` с другим значением вернёт `FALSE`)
- `<>` или `!=` – Неравенство
- `<` – Меньше
- `<=` – Меньше или равно
- `>` – Больше
- `>=` – Больше или равно
- `IS NULL` – Проверяет отсутствие значения
- `BETWEEN ( BETWEEN lower_bound AND upper_bound )` – Проверяет попадание значения в диапазон
- `IN` – Проверяет наличие значения в списке.
- `LIKE` – Проверяет соответствие строки паттерну. Маски задаются следующим образом:
 - `%` – любая последовательность символов.
 - `_` – один символ.
 - `\` – дефолтный символ для экранирования.

- Свой символ для экранирования можно задать через оператор `ESCAPE` ,
например: `SELECT item, sale FROM items WHERE sale LIKE "%!" ESCAPE "!" ;`
- `REGEXP` – Оператор для обработки строк с помощью регулярных выражений.
- Логических операторов: `NOT` , `AND` , `OR` , `XOR`
- `COALESCE(value_list)` – возвращает первый попавшийся аргумент, отличный от `NULL`.
Если же все аргументы равны `NULL`, результатом также будет `NULL`.
- Ветвления:
 - `IF ... THEN ... [ELSIF ... THEN ... ELSE ...] END IF`
 - `CASE [...] WHEN ... THEN ... ELSE ... END CASE`

Примеры:

```
SELECT
    IF number = 0 THEN
        'zero'
    ELSIF number > 0 THEN
        'positive'
    ELSIF number < 0 THEN
        'negative'
    ELSE
        'NULL'
    END IF AS number_class
FROM
    numbers

SELECT
    CASE
        WHEN number = 0 THEN
            'zero'
        WHEN number > 0 THEN
            'positive'
        WHEN number < 0 THEN
            'negative'
        ELSE
            'NULL'
        END CASE AS number_class
FROM
    numbers
```

Оператор WITH

`WITH` предоставляет способ записывать дополнительные операторы для применения в больших запросах. Эти операторы, которые также называют общими табличными

выражениями (Common Table Expressions, CTE), можно представить как определения временных таблиц, существующих только для одного запроса.

```
WITH
    regional_sales AS (
        SELECT
            region,
            SUM(amount) AS total_sales
        FROM
            orders
        GROUP BY
            region
    ),
    top_regions AS (
        SELECT
            region
        FROM
            regional_sales
        WHERE
            total_sales > (SELECT SUM(total_sales)/10 FROM regional_sales)
    )
SELECT
    region,
    product,
    SUM(quantity) AS product_units,
    SUM(amount) AS product_sales
FROM
    orders
WHERE
    region IN (SELECT region FROM top_regions)
GROUP BY
    region,
    product;
```

SELECT: Сортировка (ORDER BY)

При выполнении `SELECT` запроса, строки по умолчанию возвращаются в неопределённом порядке. Фактический порядок строк в этом случае зависит от плана соединения и сканирования, а также от порядка расположения данных на диске, поэтому полагаться на него нельзя. Для упорядочивания записей используется конструкция `ORDER BY`.

Общая структура:

```
SELECT поля_таблиц FROM наименование_таблицы
...
```

```
ORDER BY столбец_1 [ASC | DESC][, столбец_n [ASC | DESC]];
```

где ASC и DESC задают направление сортировки, по возрастанию (default) и по убыванию соответственно.

В целом как происходит сравнение очевидно, упомяну что для строк оно идёт в лексикографическом порядке, а для тернарок при ASC сначала идут NULL, потом FALSE, потом TRUE, а при DESC наоборот – NULL в конце.

SELECT: Агрегация (GROUP BY)

GROUP BY принимает на вход одну таблицу и на выход порождает тоже одну таблицу, с тем же количеством атрибутов, но эти атрибуты уже разделены на два набора. А строк GROUP BY возвращает не больше, чем во входной таблице – по одной на каждое уникальное значение кортежа группировочных атрибутов.

Двумя наборами атрибутов таблицы на выходе являются группирующие и негруппирующие атрибуты. Группирующие атрибуты – это те, что перечислены после GROUP BY, а негруппирующие – все остальные атрибуты входной таблицы.

Группирующие атрибуты остаются обычными атрибутами, и с ними далее можно работать как обычно.

А вот с негруппирующими атрибутами просто так далее по конвейеру сделать ничего не получится – ни использовать в сортировке, ни выбрать и выдать наружу. Потому что эти атрибуты содержат не одно значение, а множество. Для того чтобы со значениями негруппирующих атрибутов можно было работать, их необходимо снова превратить в скаляры с помощью агрегирующих функций:

- count() – число записей с непустым значением.
- count(DISTINCT field_nm) – число уникальных непустых значений.
- min() / max() – находят мин. / макс. в группе.
- avg() – Среднее значение в группе.
- sum() – Сумма значений группы.

Оператор AS используется для создания алиасов для столбцов или таблиц.

Пример:

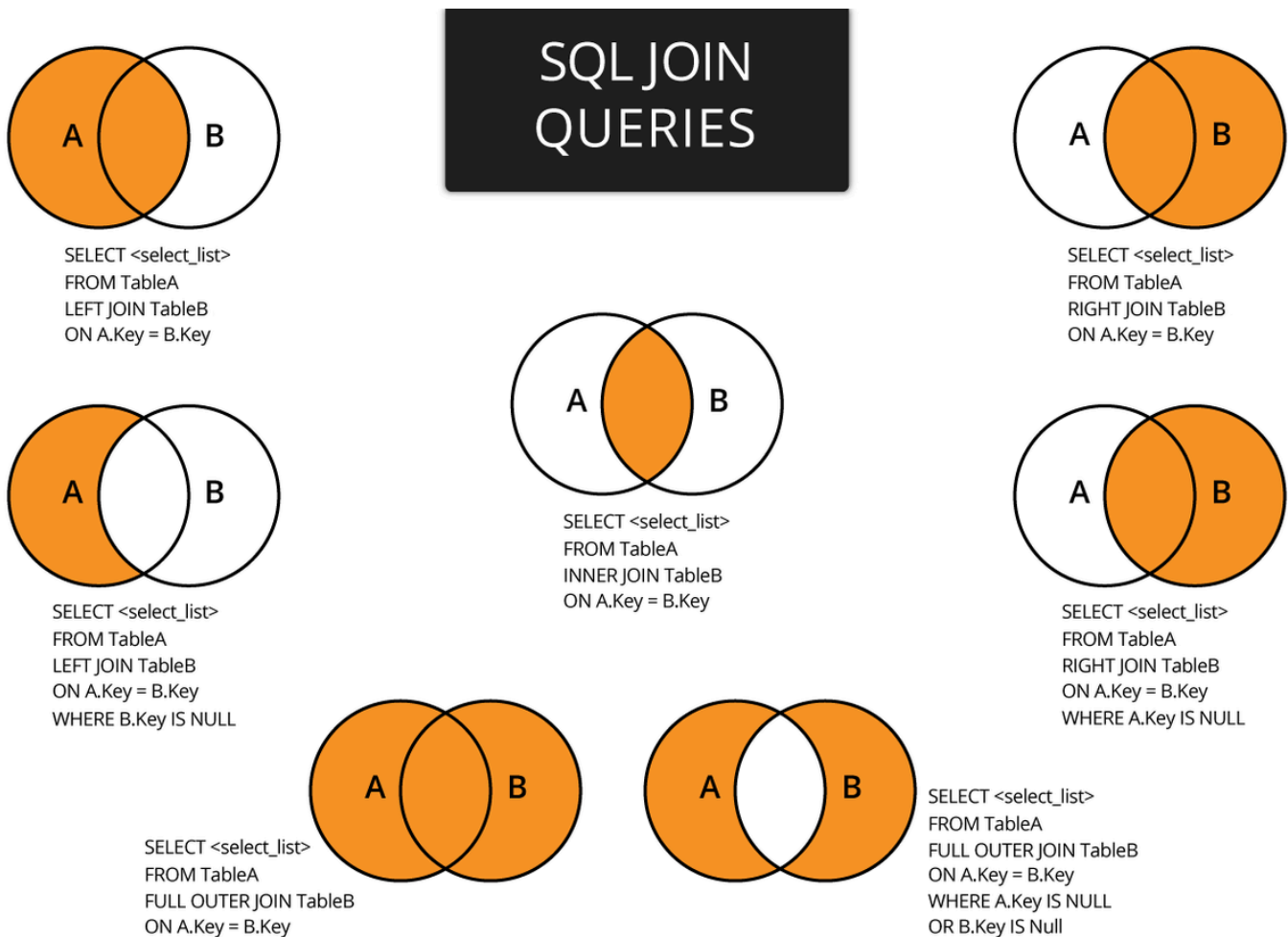
```
SELECT sex, sum(survived) AS sum_survived  
FROM titanic  
GROUP BY sex;
```

Соединение JOIN

JOIN – операция, которая комбинирует столбцы из двух или более отношений (таблиц) в одну виртуальную таблицу на основе условия связи (предиката).

Операции соединения делятся на 3 группы:

- CROSS JOIN – декартово произведение 2 таблиц
- INNER JOIN – соединение 2 таблиц по условию. В результирующую выборку попадут только те записи, которые удовлетворяют условию соединения
- OUTER JOIN – соединение 2 таблиц по условию. В результирующую выборку могут попасть записи, которые не удовлетворяют условию соединения:
 - LEFT (OUTER) JOIN – все строки "левой" таблицы попадают в итоговую выборку
 - RIGHT (OUTER) JOIN – все строки "правой" таблицы попадают в итоговую выборку
 - FULL (OUTER) JOIN – все строки обеих таблиц попадают в итоговую выборку



Периодически требуется к результату выполнения запроса добавить или исключить строки, полученные другим запросом. Для решения подобных задач в стандарте SQL предусмотрены операции над множествами:

- UNION – объединение строк (дополнение результатов первого запроса результатами второго запроса);
- INTERSECT – пересечение строк (остаются строки, присутствующие в результатах обоих запросов);
- EXCEPT – исключение строк (из строк первого запроса исключаются строки второго запроса).

Синтаксис:

```
query_1

UNION или INTERSECT или EXCEPT

query_2

...

UNION или INTERSECT или EXCEPT

query_n
```

Все запросы должны удовлетворять следующим правилам:

- количество полей во всех запросах должно совпадать;
- типы данных соответствующих полей должны совпадать.

Название поля в результате выборки берется из первого запроса.

Пример:

```
SELECT num_group, full_name
FROM aisd.submissions
GROUP BY num_group, full_name
HAVING sum(points) >= 36

EXCEPT

SELECT num_group, full_name
FROM aisd.skats
```

```
GROUP BY num_group, full_name  
HAVING count(task_name) > 2
```

3. Оконные функции в SQL: синтаксис OVER(), PARTITION BY, ORDER BY. Агрегатные функции в оконном контексте. Ранжирующие функции. Смещения. Фреймы.

Оконные функции в SQL – это функции, которые выполняют вычисления над набором строк, связанным с текущей строкой, и возвращают отдельное значение для каждой строки результирующего набора. В отличие от `GROUP BY`, оконные функции не объединяют несколько строк в одну и не уменьшают степень детализации результата. Исходные строки сохраняются, а к каждой из них добавляется вычисленное значение.

Например, с помощью оконной функции можно одновременно вывести зарплату каждого сотрудника и среднюю зарплату по его отделу:

```
SELECT  
    name,  
    department,  
    salary,  
    AVG(salary) OVER (PARTITION BY department) AS department_avg  
FROM employees;
```

Для каждой строки функция `AVG()` вычисляет среднюю зарплату по соответствующему отделу, но сами строки сотрудников при этом не объединяются.

Основные свойства оконных функций:

- возвращают значение для каждой строки результирующего набора;
- могут выполнять вычисления по всему набору строк, по отдельным секциям или как скользящее окно;
- вычисляются над набором строк, сформированным после выполнения `FROM`, `WHERE`, `GROUP BY`, агрегатных функций и `HAVING`; непосредственно использовать оконные функции можно только в списке `SELECT` и в итоговом `ORDER BY`.

note Если необходимо отфильтровать строки по результату оконной функции, используется подзапрос или общее табличное выражение.

Синтаксис оконной функции и инструкция `OVER`

Предложение `OVER` указывает, что функция должна быть выполнена в оконном контексте и задаёт описание окна, относительно которого производится вычисление.

Общий синтаксис оконной функции выглядит следующим образом:

```
function_name(arguments) OVER (  
    [PARTITION BY expressions]  
    [ORDER BY expressions]  
    [frame_clause]  
) AS attribute_name
```

В круглых скобках после `OVER` указывается описание окна. Оно может включать:

1. `PARTITION BY` – разделение набора строк на независимые секции;
2. `ORDER BY` – определение порядка строк внутри каждой секции;
3. `ROWS`, `RANGE` или `GROUPS` – определение границ фрейма.

В качестве оконных могут использоваться:

1. агрегатные функции;
2. ранжирующие функции;
3. функции смещения;
4. функции доступа к значениям внутри фрейма.

(Подробнее о них позже).

В одном запросе можно использовать несколько оконных функций с разными окнами:

```
SELECT  
    name,  
    department,  
    salary,  
    AVG(salary) OVER (PARTITION BY department) AS department_avg,  
    AVG(salary) OVER () AS company_avg  
FROM employees;
```

Если несколько оконных функций используют одинаковое описание окна, его можно вынести в отдельное предложение `WINDOW` :

```
SELECT  
    name,  
    department,  
    salary,
```



```
SUM(salary) OVER w AS department_total,  
AVG(salary) OVER w AS department_avg  
FROM employees  
WINDOW w AS (  
    PARTITION BY department  
);
```

Именованные окна уменьшают дублирование кода и упрощают сопровождение запроса.

PARTITION BY

PARTITION BY разделяет результирующий набор строк на независимые секции. Оконная функция применяется отдельно к каждой секции.

Основные свойства PARTITION BY :

- является необязательной частью описания окна;
- может использовать один или несколько столбцов;
- не объединяет строки;
- определяет независимые секции для вычисления функции;
- при отсутствии PARTITION BY весь результирующий набор считается одной секцией.

ORDER BY

ORDER BY внутри OVER определяет порядок строк в каждой секции.

Синтаксически некоторые оконные функции могут использоваться без ORDER BY , однако без явно заданного порядка результат может быть неопределённым или малоинформативным. Например, ROW_NUMBER() OVER () присвоит строкам номера, но порядок присвоения не будет гарантирован.

Для агрегатных функций ORDER BY необязателен:

Без ORDER BY агрегат обычно вычисляется по всей секции:

```
SELECT  
    name,  
    department,  
    salary,  
    SUM(salary) OVER (  
        PARTITION BY department  
    ) AS department_total  
FROM employees;
```

Одинаковая сумма отдела будет указана для каждой строки этого отдела.

При добавлении `ORDER BY` агрегатная функция может использоваться для вычисления нарастающего итога:

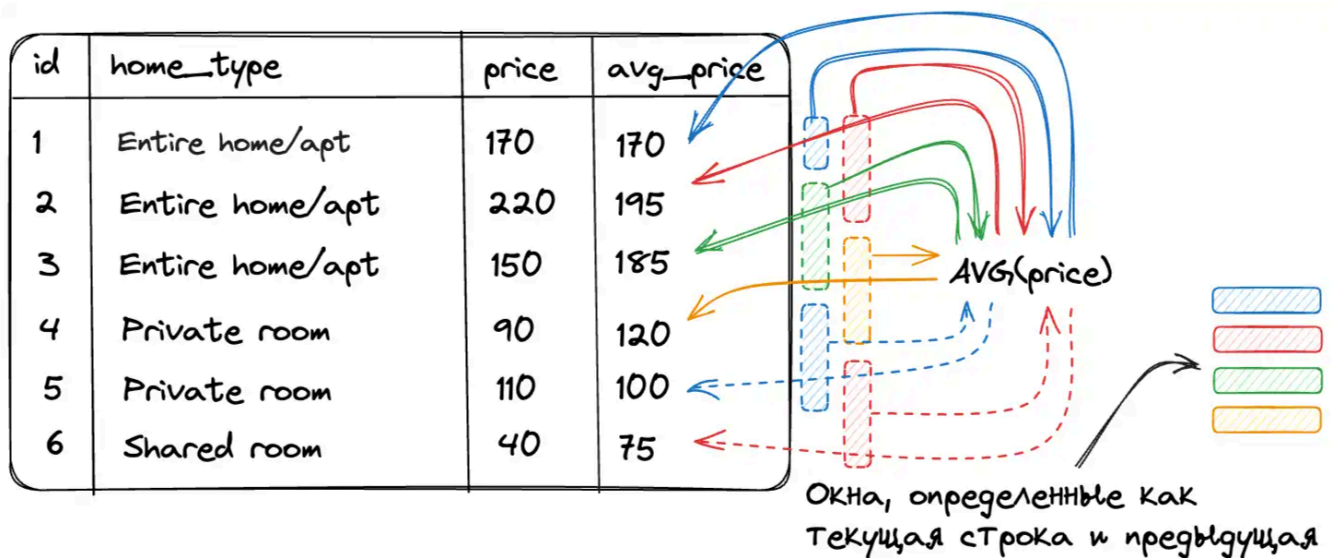
```
SELECT
    name,
    salary,
    SUM(salary) OVER (
        ORDER BY salary
        ROWS BETWEEN UNBOUNDED PRECEDING
            AND CURRENT ROW
    ) AS running_total
FROM employees;
```

Здесь для каждой строки рассчитывается сумма от начала набора до текущей строки.

Фреймы

Фрейм – это подмножество строк текущей секции, которое используется при вычислении оконной функции для текущей строки.

Секция определяется с помощью `PARTITION BY`, а фрейм ограничивает количество строк внутри этой секции.



Общий синтаксис определения фрейма:

```
ROWS | RANGE | GROUPS
BETWEEN frame_start AND frame_end
```

Начальная и конечная границы могут задаваться следующими конструкциями:

- `CURRENT ROW` — текущая строка или текущая группа равноправных строк;
- `N PRECEDING` — определённое количество строк, групп или единиц значения до текущей позиции;
- `N FOLLOWING` — определённое количество строк, групп или единиц значения после текущей позиции;
- `UNBOUNDED PRECEDING` — начало текущей секции;
- `UNBOUNDED FOLLOWING` — конец текущей секции.

Режимы `ROWS`, `RANGE` и `GROUPS` по-разному определяют границы фрейма:

`ROWS BETWEEN`

`ROWS` определяет границы фрейма по физическому положению строк.

Пример:

```
-- Вычисление скользящего среднего

SELECT
    year,
    month,
    expense,
    ROUND(AVG(expense) OVER w, 2) AS rolling_avg
FROM expenses
WINDOW w AS (
    ORDER BY year, month
    ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
)
ORDER BY year, month;
```

Для каждого месяца среднее рассчитывается по расходам предыдущего, текущего и следующего месяцев.

Для первой строки предыдущей строки нет, поэтому в расчёте участвуют только текущая и следующая строки. Для последней строки аналогично используются текущая и предыдущая строки.

`GROUPS BETWEEN`

`GROUPS` определяет границы фрейма по группам строк, имеющих одинаковую позицию с точки зрения оконной сортировки `ORDER BY`.

Пример:

```
-- Считаем количество записей нарастающим итогом.
```

```
SELECT
    id,
    name,
    department,
    COUNT(*) OVER w AS cnt
FROM employees
WINDOW w AS (
    ORDER BY department
    GROUPS BETWEEN UNBOUNDED PRECEDING
            AND CURRENT ROW
)
ORDER BY department, id;
```

RANGE BETWEEN

RANGE определяет фрейм на основе диапазона значений выражения, указанного в ORDER BY .

Пример:

```
-- Посмотрим для каждой гитары количество альтернатив с ценой в диапазоне +-100$ от
текущей и их средний рейтинг.
```

```
SELECT
    model,
    type,
    price,
    rating,
    COUNT(*) OVER w AS alternatives_count,
    ROUND(AVG(rating) OVER w, 2) AS alternatives_avg_rating
FROM guitars
WINDOW w AS (
    PARTITION BY type
    ORDER BY price
    RANGE BETWEEN 100 PRECEDING AND 100 FOLLOWING
    EXCLUDE CURRENT ROW
);
```

EXCLUDE CURRENT ROW исключает текущую строку из фрейма. При этом другие строки с такой же ценой могут оставаться во фрейме.

Функции в оконном контексте

Агрегирующие функции

Агрегатные функции вычисляют обобщённое значение по набору строк.

Функция	Описание
MIN(value)	Возвращает минимальное значение среди строк текущей секции или фрейма
MAX(value)	Возвращает максимальное значение
COUNT(value)	Подсчитывает количество значений, не равных NULL
COUNT(*)	Подсчитывает количество строк
AVG(value)	Вычисляет среднее значение, игнорируя NULL
SUM(value)	Вычисляет сумму значений
STRING_AGG(value, separator)	Объединяет значения в строку с указанным разделителем

Ранжирующие функции

Ранжирующие функции присваивают строкам номера или ранги в соответствии с порядком, заданным в оконном ORDER BY .

Основные ранжирующие функции:

Функция	Описание
ROW_NUMBER()	Присваивает каждой строке уникальный последовательный номер
RANK()	Присваивает одинаковый ранг строкам с одинаковыми значениями сортировки и оставляет пропуски после совпадений
DENSE_RANK()	Присваивает одинаковый ранг строкам с одинаковыми значениями, но не оставляет пропусков
NTILE(n)	Распределяет строки по n группам и возвращает номер группы

Пример:

```
SELECT
  ROW_NUMBER() OVER w AS row_number,
  DENSE_RANK() OVER w AS dense_rank,
  RANK() OVER w AS rank,
  name,
  department,
  salary
```

```
FROM employees
WINDOW w AS (
    ORDER BY salary DESC
)
ORDER BY salary DESC;
```

Результат может выглядеть следующим образом:

row_number	dense_rank	rank	name	department	salary
1	1	1	Иван	it	120
2	2	2	Леонид	it	104
3	2	2	Марина	it	104
4	3	4	Анна	sales	100
5	4	5	Вероника	sales	96
6	4	5	Григорий	sales	96
7	5	7	Ксения	it	90
8	6	8	Елена	it	84
9	7	9	Борис	hr	78
10	8	10	Дарья	hr	70

note `ntile(n)` в случае, когда количество строк в окне `k` не кратно `n`, в первые несколько ($k \bmod n$) групп помещает `n+1` строку, а в остальные — `n`.

Функции смещения

Функции смещения позволяют получать значения из предыдущих или следующих строк текущей секции.

Функция	Описание
<code>LAG(value, offset, default)</code>	Возвращает значение из предыдущей строки с указанным смещением
<code>LEAD(value, offset, default)</code>	Возвращает значение из следующей строки с указанным смещением

- Параметр `offset` определяет величину смещения и по умолчанию равен `1`.
- Параметр `default` определяет значение, которое будет возвращено, если соответствующей строки не существует. По умолчанию возвращается `NULL`.

Пример:

```
-- Для каждой строки будет выведена выручка предыдущего месяца.
```

```
SELECT
    month,
    revenue,
    LAG(revenue) OVER (
        ORDER BY month
    ) AS previous_revenue
FROM monthly_revenue;
```

```
-- Можно сразу вычислить разницу:
```

```
SELECT
    month,
    revenue,
    revenue - LAG(revenue) OVER (
        ORDER BY month
    ) AS revenue_change
FROM monthly_revenue;
```

Функции доступа к значениям фрейма

Следующие функции возвращают значения из определённых строк текущего фрейма:

Функция	Описание
FIRST_VALUE(value)	Возвращает значение из первой строки текущего фрейма
LAST_VALUE(value)	Возвращает значение из последней строки текущего фрейма
NTH_VALUE(value, n)	Возвращает значение из n -й строки текущего фрейма

Пример:

```
SELECT
    name,
    department,
    salary,
    FIRST_VALUE(salary) OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS highest_salary
FROM employees;
```

Поскольку строки внутри отдела отсортированы по убыванию зарплаты, `FIRST_VALUE()` возвращает максимальную зарплату в отделе.

note При использовании `LAST_VALUE()` необходимо учитывать границы фрейма. Если фрейм не задан явно, функция может вернуть значение текущей строки или последней строки, равной ей с точки зрения оконной сортировки.